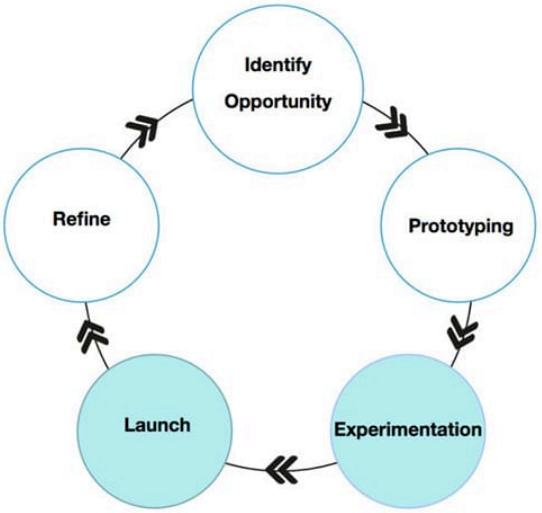
Building an Intelligent Experimentation Platform with Uber Engineering

uber.com/en-PL/blog/experimentation-platform

Eva Feng May 4, 2017



During our inaugural Uber Technology Day, data scientist Eva Feng delivered a presentation on Uber's experimentation platform (XP). In this article, she and colleague Zhenyu Zhao detail how Uber engineered an XP capable of rolling out new features stably and quickly at scale.

The lifecycle of feature development for a mobile app is composed of identifying opportunities, prototyping, experimentation, launching, refining, and identifying opportunities again. Experimentation is a critical stage of the product lifecycle; it is the process of discovering and determining whether or not new features are successful. Given Uber's hypergrowth, the goal of our XP is to ensure that new features roll out successfully and then return actionable analysis.

Uber's XP is unique in the industry because we launch not just experiments but also feature releases on a nationwide and even global scale which can almost immediately improve how people (and now food) get around in the physical world.

Initially, this massive scale across both products and markets made it challenging to build an XP that multiple teams with varying programming backgrounds and preferences could understand and use. Today, many teams at Uber use the XP to deploy features for their products, including the rider, driver, and UberEATS apps.

In this article, we will discuss the challenges and opportunities faced when developing our XP's two primary components—a staged rollout and an intelligent analysis tool—as well as our finished product's achieved outcomes.

All the World is a Stage(d Rollout)

The first element of the experimentation lifecycle at Uber is a staged rollout, the process of deploying a feature first to a small portion of users and then gradually ramping up to stages with larger proportions. Eventually, we reach 100 percent of all users that fall under a target specification (for instance, geographic location, which can be as small as a district of a city or as large as the entire world).

The goal of a staged rollout is to make the deployment of new features as stable and reliable as possible by controlling user exposure in the early stages and monitoring the impact of the feature on key business metrics at each stage. A staged rollout is different from standard A/B testing in many ways, as outlined in the table below:

Differences Between a Staged Rollout and Standard A/B Testing

	Staged Rollout	**Standard A/B Testing**
Type of Process	Reliable feature release	Feature evaluation
Goal	Whether the feature is causing a regression	Whether the feature is successful
Experiment Design and Configuration	Multiple adaptive stages	One fixed stage
Metrics	Core app health and business metrics	Complete set of app metrics
Statistical Test	Sequential test	Fixed horizon test (e.g. t-test, chi-squared test, and bootstrapping)

Feature rollout can be risky if the target population of the rollout is large and there is a lack of effective monitoring to measure the impact of the feature on key business metrics.

Architecting the New Feature Rollout System

The XP we developed has two components: a staged rollout configuration process and a new monitoring algorithm. To control the user's risk of exposure to error, we stage the rollout process to be a gradual ramp-up. To enable more efficient and precise rollout monitoring, we use an algorithm which enables us to continuously monitor the impact of the feature on key metrics while controlling false positive rates.

The null hypothesis is that the new feature (treatment) does not have a negative effect on the key metrics. For instance, rather than reporting the raw number of trips from a high level, the algorithm identifies if the differences in trip rate (defined as the proportion of sessions containing a trip) are significant between the control (feature-disabled) and treatment (feature-enabled) groups.

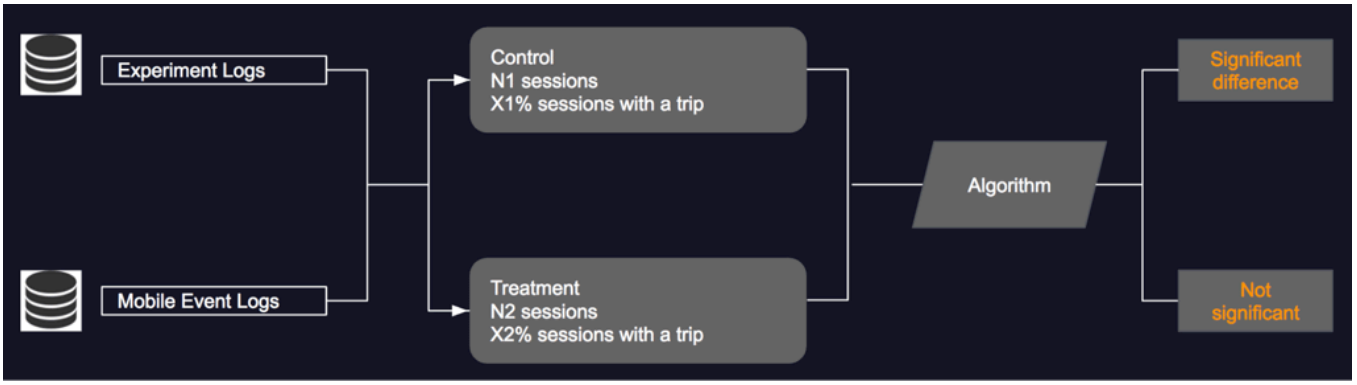


Figure 1: The rollout analytics algorithm uses event log information to identify outages.

To determine if the difference between these metrics was significant when continuously monitored, we experimented with using three different types of tests, a t-test, a sequential likelihood ratio test (SLRT), and a delete-a-group jackknife variance estimation, in our algorithm.

The first test we tried using was a t-test. Our t-test produced an inflated false positive rate of 50 percent accuracy at a confidence level of 95 percent. This outcome was expected since a t-test, like other fixed-horizon tests, is not designed for continuous monitoring.

Next, we decided to adjust our algorithm to perform an SLRT with an independence assumption in the hope of accurately detecting regression. While more accurate than the t-test model, the original format of the SLRT gave a false positive rate of just over 30 percent. Since our metrics are conducted at the session level, it was problematic that the behaviors of two independent sessions were highly correlated with each other when these sessions came from the same user.

The final test we tried was a sequential likelihood ratio test using the delete-a-group jackknife variance estimation. We found that this test provided us with a 5 percent false positive rate when continuously monitored, which was sufficient for our needs.

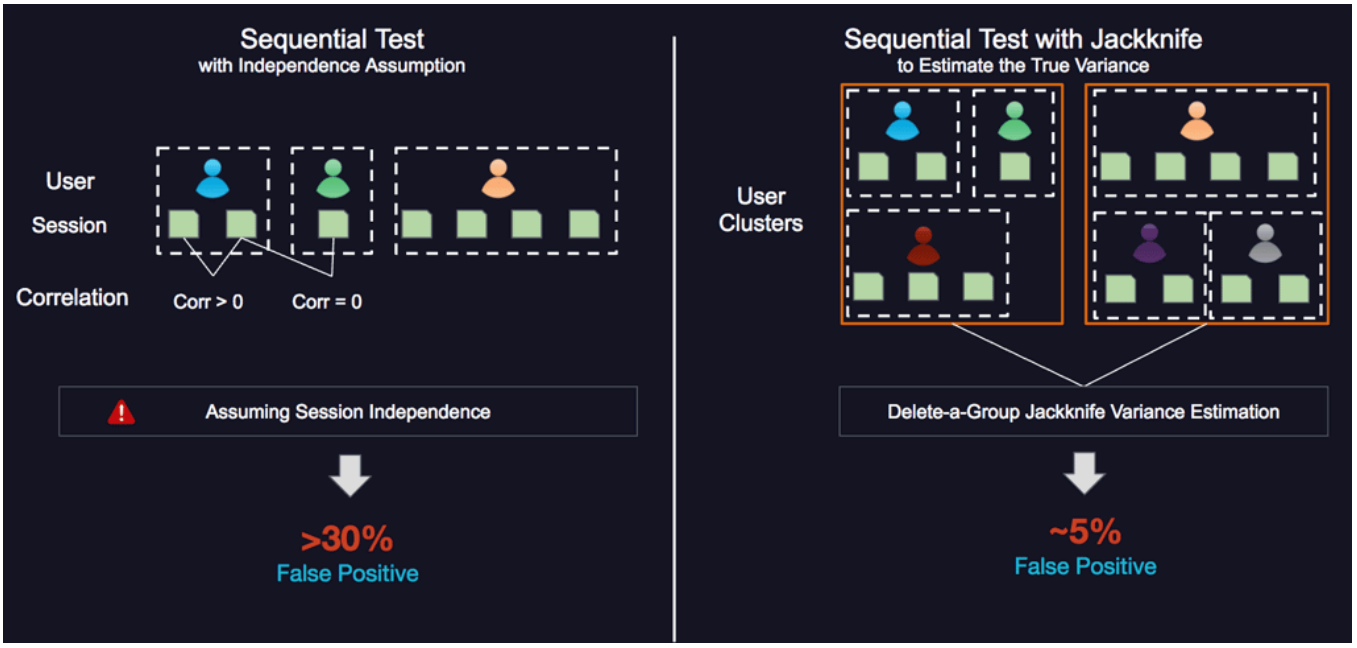


Figure 2: At 5 percent, an SLRT using the delete-a-group jackknife variance resulted in a far lower rate of false positives than the sequential likelihood ratio test with independence assumption.

Achievement and Outcomes

Since our initial testing, this staged rollout framework has proven indispensable to many feature deployments at Uber. It has been widely adopted by different product teams and has caught regressions caused by features at a low rollout stage, thereby reducing their impact on users.

One such regression occurred during a login feature rollout that led to an outage in one small population of users. A new feature that allows users to login using their phone numbers as opposed to their usernames was deployed in a single city as a staged rollout. When we deployed this new feature for app users in this geographic area, we noticed a drop in their trip rates; through some investigation, we found many users in the treatment group were not able to successfully sign in to the app using their phone numbers. Thankfully, our XP's staged analysis functionality was able to detect this before it was deployed to all of our users in the country and prevent a full-blown outage.

Intelligent Analysis Tool

As Uber continues to expand to new cities and grow its operations in existing ones, the need to deliver real-time experiment results has become more important than ever before. For instance, city operation management teams frequently use the XP to fine tune new features, including testing out new SMS text messages and tailoring the driver onboarding experience for each city.

Due to the difficulties of working in different time zones, it previously took around a few days for the city operation managers to collaborate with the XP team to run an analysis report following a rollout. With our new framework in place, we have been able to cut down on execution time by a magnitude of days, delivering the experiment results to city operation teams no matter their location with very short latency.

Defining the XP Architecture

To optimize the speed of our analytics, we initially pre-computed the statistical value of our business metrics in Hive. However, this method of computing statistical values did not afford our end users enough flexibility to customize the definitions of their metrics.

The new analysis tool does not pre-compute the data of the metrics, which cut down on our data storage spend and reduced our analysis generation time. Now, when the data is ready for analysis, we use a SQL query file to generate metrics on the fly whenever people make a request on the WebUI. After that, we use Scala as our service engine to compute the probability (p-value) that the treatment group mean is significantly different than the control group mean, determining if the experiment reached the target sample size.

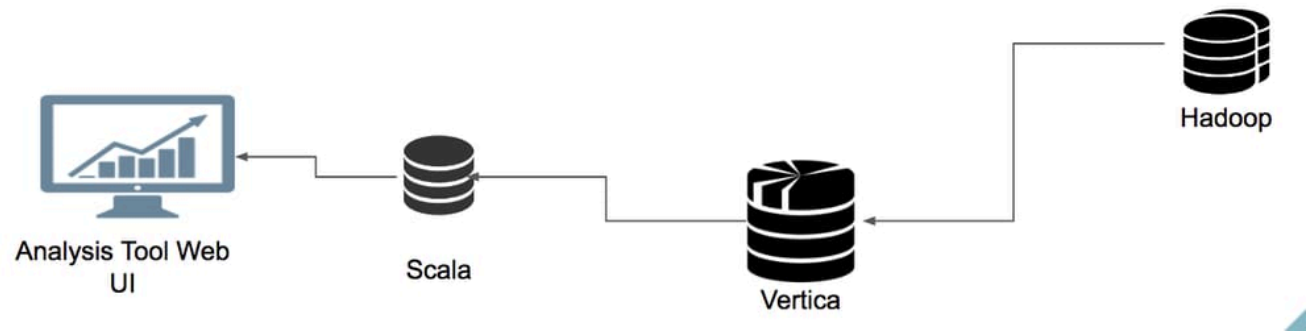


Figure 3: The new XP analysis tool is supported by a robust backend including multiple databases and service engines.

The Science Behind the XP

Uber's new intelligent analysis tool conducts post-experiment analysis, which uses a different methodology than the staged rollout use case outlined earlier. For the purposes of this tool, we classified hundreds of metrics into three categories: proportion metrics, continuous metrics, and ratio metrics. We also collaborated with Uber's experimentation and research team to design methods to evaluate if the treatment group provides more significant lift when compared to the control group.

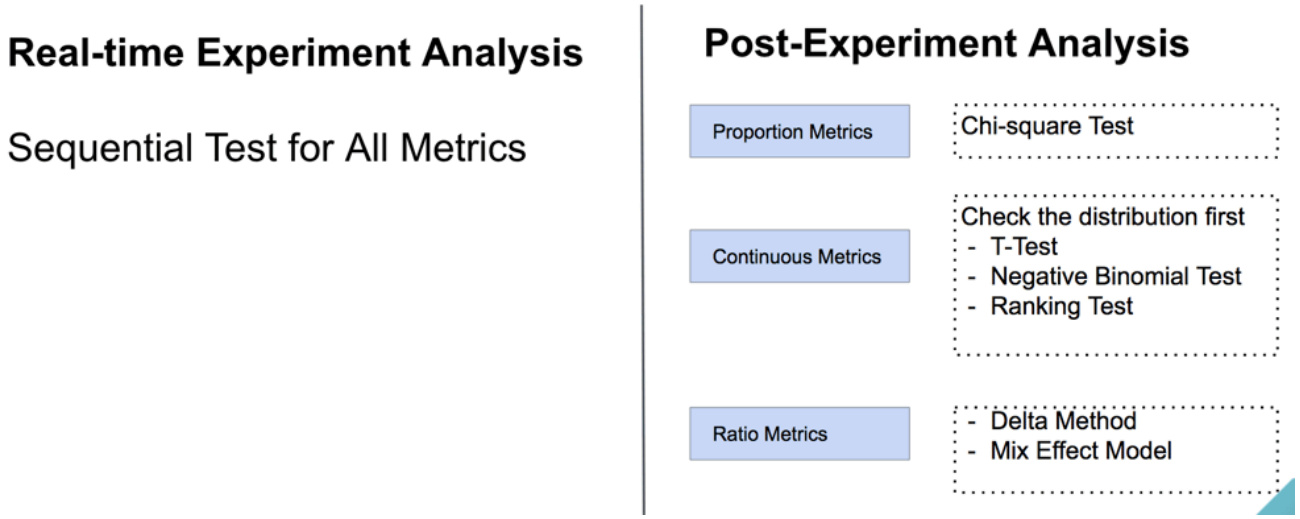


Figure 4: An overview of methodology used in Uber's XP analysis tool.

Our Uber XP team is dedicated to improving the stability of our apps through our staged rollout process and our post-experiment analysis tool. In 2017 and beyond, we have much more experimentation ahead of us on our way to making Uber even more effective and useful to drivers and riders in the real world. So if you are interested in being a data scientist on the Experimentation Platform team at Uber, check out our Careers Page for openings. The more we know, the more we know we have to do!

Watch on YouTube

Eva Feng and Zhenyu Zhao are data scientists on Uber's Experimentation Platform team.

Zhenyu Zhao is a senior data scientist with Uber's Platform Data Science team.